

Principles of Programming Languages, 2025.06.16

Important notes

- Total available time: 2h (*multichance* students do not need to solve Exercise 3).
- You may use any written material you need, and write in English or in Italian.
- You cannot use electronic devices during the exam: every phone must be turned off and kept on your table.
- You cannot use library functions not covered in class in your code.

Exercise 1, Scheme (11 pts)

We want to implement a version of call/cc, called *store-named-cc*, where the continuation is given a *name* (a symbol) and is implicit, i.e. we do not need to pass a variable to the construct to store the continuation, e.g. (store-named-cc 'stuff (+ x 1)), where *stuff* is the name that we want to use for the continuation. To run the continuation, we can use the associated construct *run-named-cc name*, which may take other parameters, if the continuation requires them.

Exercise 2, Haskell (11 pts)

Consider the following datatype definition.

```
data PTL a = P a a | T a a a | L [PTL a] deriving (Show, Eq)
```

Describe its possible usage, then make PTL an instance of Functor, Foldable, and Applicative.

Exercise 3, Erlang (11 pts)

1) Write a function: *deepmaprev(F, List)*, that applies a function F to every non-list element in a possibly deep list, and reverses the list structure at every level.

Example:

```
> deepmaprev(fun(X) -> X+1 end, [[0,1],2,3,[4,[5]]]).  
[[[6],5],4,3,[2,1]]
```

2) Write a parallel version, that behaves exactly like *deepmaprev/2* but spawns a separate process to handle each top-level element in the input list, then collects and assembles the results in the correct order.

Solutions

Ex 1

```
(define *stored-named-cc* (make-hash))

(define-syntax store-named-cc
  (syntax-rules ()
    ((_ name e ...)
     (call/cc (lambda (k)
                 (display 'hello)
                 (hash-set! *stored-named-cc* name k)
                 e ...))))))

(define (run-named-cc name . v)
  (let ((k (hash-ref *stored-named-cc* name)))
    (apply k v)))
```

Ex 2

-- PTL is a sum type of a pair, a triple, and a recursive list of PTL. It could be used to store complex tree-like structures.

```
instance Functor PTL where
  fmap f (P x y) = P (f x) (f y)
  fmap f (T x y z) = T (f x) (f y) (f z)
  fmap f (L xs)   = L (map (fmap f) xs)

instance Foldable PTL where
  foldr f z (P x y)      = f x (f y z)
  foldr f z (T x y w)    = f x (f y (f w z))
  foldr f z (L xs)       = foldr (\ptl acc -> foldr f acc ptl) z xs

instance Applicative PTL where
  pure x = P x x

  (P f g) <*> (P x y)      = P (f x) (g y)
  (T f g h) <*> (T x y z) = T (f x) (g y) (h z)
  (L fs) <*> (L xs)        = L (apList fs xs)
  where
    apList [] _ = []
    apList _ [] = []
    apList (f:fs) (x:xs) = (f <*> x) : apList fs xs
```

Ex 3

```
deepmaprev(F, L) when is_list(L) ->
  lists:reverse([deepmaprev(F, X) || X <- L]);
deepmaprev(F, X) ->
  F(X).

deepmaprev_par(F, L) when is_list(L) ->
  Parent = self(),
  Pids = [spawn(fun() ->
                  Result = deepmaprev(F, X),
                  Parent ! {self(), Result}
                end) || X <- L],
  gather_results(Pids, []).

gather_results([], Acc) ->
  Acc;
gather_results([Pid | Rest], Acc) ->
  receive
    {Pid, Result} ->
      gather_results(Rest, [Result | Acc])
  end.
```